

QLoRA: Quantised LoRA

4-bit quantisation of the frozen base model + LoRA adapters: fine-tune 65B on 48 GB

10 min

Duration

B3 – Apply

Bloom's

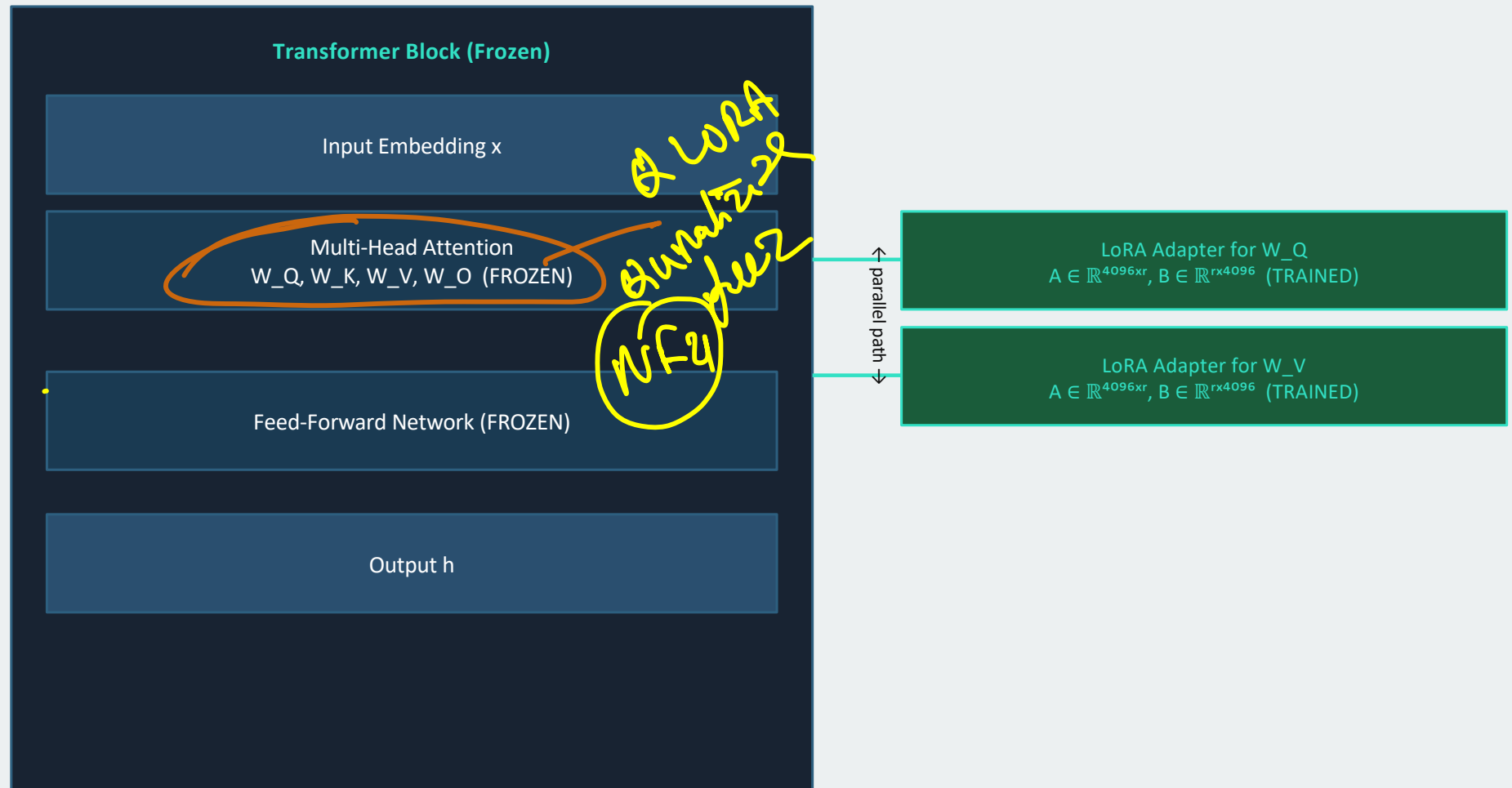
L3 – Advanced

Level

AI-FT-IN-TH-000007

ID

LoRA Architecture — Where Adapters Are Inserted



QLoRA — Definition & Key Innovation

QLoRA (Dettmers et al., NeurIPS 2023)

QLoRA extends LoRA by quantising the frozen base model weights to 4-bit precision using the NF4 (Normal Float 4-bit) data type. LoRA adapters are trained in bf16 on top of the frozen 4-bit base. Gradients flow through the quantised weights only to the LoRA parameters.

- Two novel techniques: (1) NF4 quantisation — information-theoretically optimal for normally distributed weights; (2) double quantisation — quantises the quantisation constants themselves
- Memory saving: 65B model = 130 GB in fp16 → 35 GB in NF4 + LoRA → fits on a single A100 48 GB
- Quality: QLoRA fine-tunes match full-precision LoRA fine-tunes on most benchmarks — negligible accuracy loss

De facto standard: QLoRA is the default fine-tuning approach on consumer hardware (RTX 3090/4090, A10G)

FP16	Int4	NF4
$w_1: 0.1$	0	0.1
$w_2: 0.05$	0	0.1
$w_3: 0.56$	1	0.5
$w_4: 0.9$	1	1

$2^4 = 16$ locations

$\{-7, \dots, 0, \dots, 8\}$

$\{1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1\}$

$\{-0.1, 0.0, 0.1, \dots, 1\}$

QLoRA Memory Savings — 65B Model Worked Example

Scenario: Fine-tune LLaMA 65B (r=16, targets q_proj & v_proj)

Without QLoRA (standard LoRA in bf16):

Base weights (bf16): $65B \times 2 \text{ bytes} = 130 \text{ GB}$ ← doesn't fit on any single GPU

LoRA params (bf16): $\sim 200M \times 2 \text{ bytes} \approx 400 \text{ MB}$

Total $\approx 130.4 \text{ GB}$ → Requires 2× A100 80GB at minimum

With QLoRA (NF4 quantisation + double quantisation):

Base weights (NF4): $65B \times 0.5 \text{ bytes} = 32.5 \text{ GB}$ (4-bit)

Double quant savings: -2 GB (quant constants compressed)

LoRA adapters (bf16): $\sim 400 \text{ MB}$

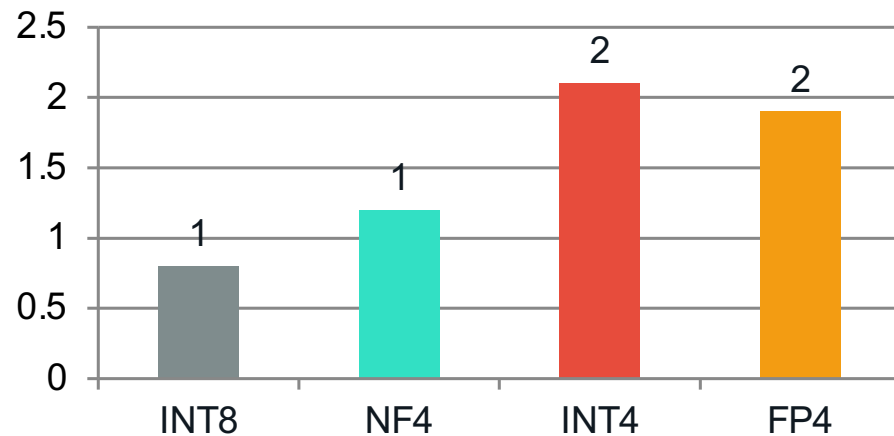
Total $\approx 31 \text{ GB}$ → Fits on a single A100 40GB or 2× RTX 4090!

NF4 Data Type — Why It's Optimal for LLM Weights

Key Insight

LLM weight matrices have approximately normally distributed values. NF4 is designed so that each of its 16 quantisation bins represents an equal fraction of the normal distribution — maximising the information stored per bit. Standard 4-bit integer (INT4) wastes capacity because weight distributions are not uniform.

Quantisation Error by Data Type



Double Quantisation

- Standard 4-bit: quantisation constants stored in fp32 (0.5 bytes per weight average)
- Double quant: quantise the constants too — reduce constant footprint from 32 bits to 8 bits
- Additional saving: ~0.37 bits per parameter = ~2.9 GB for 65B model

QLoRA Setup — BitsAndBytesConfig + SFTTrainer

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTTrainer
import torch

# Step 1: 4-bit quantisation config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NF4 data type
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,    # double quantisation
)

# Step 2: Load model in 4-bit
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8B", quantization_config=bnb_config, device_map="auto"
)
model = prepare_model_for_kbit_training(model) # enable gradient checkpointing

# Step 3: Add LoRA adapters on top of quantised model
lora_config = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"],
                        lora_dropout=0.05, bias="none", task_type="CAUSAL_LM")
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# → trainable params: 8,388,608 || all params: 8,038,522,880 || 0.1044%
```

`prepare_model_for_kbit_training()` enables gradient checkpointing and casts norms to fp32 — both essential for stable 4-bit training.

QLoRA vs LoRA vs Full Fine-Tuning — Memory & Quality

Memory Requirements

- Full FT (7B): ~112 GB VRAM
- LoRA (7B, bf16): ~18 GB VRAM
- QLoRA (7B, nf4): ~7–8 GB VRAM ✓
- QLoRA (13B): ~12 GB VRAM ✓
- QLoRA (65B): ~31 GB VRAM ✓
- QLoRA (70B): ~35 GB VRAM ✓

Quality Comparison

- Full FT → baseline (100%)
- LoRA (r=16) → 99.2% of full FT quality
- QLoRA (nf4, r=16) → 98.8% of full FT
- Quality drop from 4-bit quantisation: negligible on most benchmarks
- Recommendation: Use QLoRA by default on consumer hardware
- Use full LoRA (bf16) only when compute budget allows

KEY REFERENCES

- Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. arxiv.org/abs/2305.14314
- Dettmers, T. et al. (2022). LLM.int8(): 8-bit Matrix Multiplication. NeurIPS 2022.
- HuggingFace bitsandbytes integration. huggingface.co/docs/transformers/quantization/bitsandbytes

 **Next: Concept 9: Prefix Tuning (AI-FT-IN-TH-000008)**